

load-tracker-cookies

scripts/load-tracker-cookies.sh — Tracker Cookies Autoload

Revision: 2 **Last modified:** 2026-08-18T00:00:00Z **Purpose:** External operator guide for the per-tracker cookies autoload primitive. **Last verified:** 2026-08-18

Overview

scripts/load-tracker-cookies.sh is boba's **single primitive** for turning a per-tracker Netscape cookies.txt file exported from a browser into the <TRACKER>_COOKIES=... env var that the download-proxy container consumes for authenticated tracker access.

It implements the operator mandate (2026-08-15):

“System seeks for cookies txt files in naming convention cookies_rutracker.txt — cookies_<TRACKER>.txt and loads data from them for trackers which need them. Let's use by default host machine home directory → Downloads folder. So we MUST support env. vars and use of cookies files.”

The convention is fixed and boring on purpose:

```
${TRACKER_COOKIE_DIR:-$HOME/Downloads}/cookies_<tracker>.txt
```

For every file present it:

1. Parses the Netscape TSV, keeping only cookies whose domain belongs to that tracker (leading . stripped, case-insensitive substring match), deduplicates by cookie name.
2. Runs the mandatory **§11.4.10.A pre-store leak audit** (git grep -F on tracked files + git log -S) — any cookie value already present in the working tree or recent history **blocks the store** and the operator is told to rotate.
3. Writes the resulting
<TRACKER_UPPER>_COOKIES='<name=value; ...>' line into .env

atomically (temp + chmod 600 + mv) — the same seam boba-jackett uses. .env stays 0600 and gitignored.

4. Is **idempotent** — if the parsed value already matches what .env holds, it skips the write.
5. **Never** logs cookie values — only tracker name, count, cookie names, and file mtime.

The loader is called automatically before every boba-svc up, boba-svc restart, install.sh Stage 6, and start.sh boot — so re-exporting a fresh cookie file in your browser is all the operator has to do; the containers pick it up on the next restart.

Prerequisites

- bash 4+, coreutils, git (for the leak audit)
- scripts/extract-tracker-cookies.sh (delegated primitive for rutracker / nnmclub / iptorrents; inline extraction is used for the other trackers)
- The cookies_<tracker>.txt file(s) exported from a logged-in browser session in Netscape TSV format (Firefox: Cookie Quick Manager add-on → *Export cookies to file*. Chrome: Get cookies.txt extension).

Usage examples

Example 1 — one-shot dry run against ~/Downloads

Verified output (2026-08-15, 4 cookie files present):

```
$ bash scripts/load-tracker-cookies.sh --dry-run
[extract] rutracker: 4 cookie(s) for own domain — names: bb_guid
bb_session bb_ssl cf_clearance
[load-tracker-cookies] rutracker: WOULD LOAD (4 cookie(s)) into /
run/media/milosvasic/DATA4TB/Projects/boba/.env as
RUTRACKER_COOKIES — DRY-RUN (no write)
[extract] nnmclub: 9 cookie(s) for own domain — names:
eb927f21fc_blockTimer eb927f21fc_delayCount phpbb2mysql_4_data
phpbb2mysql_4_sid u_eb927f21fc uuid _ym_d _ym_isad _ym_uid
[load-tracker-cookies] nnmclub: WOULD LOAD (9 cookie(s)) into /
run/media/milosvasic/DATA4TB/Projects/boba/.env as NNMCUB_COOKIES
— DRY-RUN (no write)
[load-tracker-cookies] rutor: 12 cookie(s) for own domain — names:
domain_sid ec592524fc_blockTimer ec592524fc_delayCount
_ohmybid_cmh redir_ipv6 _sltb _sltm u_ec592524fc _ym_d _ym_isad
_ym_uid _ym_visorc
[load-tracker-cookies] rutor: WOULD LOAD (12 cookie(s)) into /run/
media/milosvasic/DATA4TB/Projects/boba/.env as RUTOR_COOKIES —
DRY-RUN (no write)
[load-tracker-cookies] kinozal: 5 cookie(s) for own domain —
```

```
names: eb3299ed2c_blockTimer eb3299ed2c_delayCount pass
u_eb3299ed2c uid
[load-tracker-cookies] kinozal: WOULD LOAD (5 cookie(s)) into /
run/media/milosvasic/DATA4TB/Projects/boba/.env as KINOZAL_COOKIES
– DRY-RUN (no write)
[load-tracker-cookies] summary: loaded=4 unchanged=0 absent=0
blocked=0 parse_err=0 (dir=/home/milosvasic/Downloads)
```

iptorrents support was added 2026-08-18 (Task #39), routed through the same extract-tracker-cookies.sh extractor as rutracker/nnmclub (required session-cookie canary: pass=). Verified output against a real, already- partially-loaded host (2026-08-18):

```
$ bash scripts/load-tracker-cookies.sh --dry-run
[extract] iptorrents: 2 cookie(s) for own domain – names: pass uid
[load-tracker-cookies] iptorrents: WOULD LOAD (2 cookie(s)) into /
run/media/milosvasic/DATA4TB/Projects/boba/.env as
IPTORRENTS_COOKIES – DRY-RUN (no write)
[load-tracker-cookies] summary: loaded=1 unchanged=4 absent=0
blocked=0 parse_err=0 (dir=/home/milosvasic/Downloads)
```

Example 2 — real write

```
$ bash scripts/load-tracker-cookies.sh
[load-tracker-cookies] rutracker: 4 cookie(s) – LOADED into .env
as RUTRACKER_COOKIES
[load-tracker-cookies] nnmclub: 9 cookie(s) – LOADED into .env
as NNMCCLUB_COOKIES
[load-tracker-cookies] rutor: 12 cookie(s) – LOADED into .env
as RUTOR_COOKIES
[load-tracker-cookies] kinozal: 5 cookie(s) – LOADED into .env
as KINOZAL_COOKIES
[load-tracker-cookies] summary: loaded=4 unchanged=0 absent=0
blocked=0 parse_err=0
```

Example 3 — idempotent re-run (nothing changed)

```
$ bash scripts/load-tracker-cookies.sh
[load-tracker-cookies] rutracker: 4 cookie(s) – UNCHANGED
(idempotent)
[load-tracker-cookies] nnmclub: 9 cookie(s) – UNCHANGED
(idempotent)
[load-tracker-cookies] rutor: 12 cookie(s) – UNCHANGED
(idempotent)
[load-tracker-cookies] kinozal: 5 cookie(s) – UNCHANGED
(idempotent)
[load-tracker-cookies] summary: loaded=0 unchanged=4 absent=0
blocked=0 parse_err=0
```

Example 4 — a single tracker only

```
$ bash scripts/load-tracker-cookies.sh --only rutor --verbose
[load-tracker-cookies] rutor: /home/milosvasic/Downloads/
cookies_rutor.txt (mtime: 2026-08-15 13:56:29)
[load-tracker-cookies] rutor: 12 cookie(s) for own domain – names:
domain_sid ec592524fc_blockTimer ...
[load-tracker-cookies] rutor: 12 cookie(s) – LOADED into .env as
RUTOR_COOKIES
[load-tracker-cookies] summary: loaded=1 unchanged=0 absent=0
blocked=0 parse_err=0
```

Example 5 — custom source directory

```
$ bash scripts/load-tracker-cookies.sh --dir /media/usb/browser-export
```

Example 6 — env-var equivalent

```
$ TRACKER_COOKIE_DIR=/media/usb/browser-export bash scripts/load-tracker-cookies.sh
```

--dir wins when both are set.

Flags

Flag	Behaviour
--dry-run	Report what WOULD change; touch nothing. Safe to run any time.
--dir <path>	Override the cookie source dir (default \$TRACKER_COOKIE_DIR else \$HOME/Downloads).
--only <tracker>	Process a single tracker (rutracker nmclub rutor kinozal iptorrents). May be repeated.
--verbose / -v	Extra progress (per-file mtime, per-cookie count). Cookie values are still never printed.
-h / --help	Print help.

Env vars

Var	Effect
TRACKER_COOKIE_DIR	Overrides the default \$HOME/ Downloads source dir. --dir wins.
BOBA_ENV_FILE	Overrides the default <repo- root>/.env write target — used by tests.

Exit codes

Code	Meaning
0	Success — every file loaded, unchanged, or absent (no file at all is a legitimate outcome).
1	\$11.4.10.A leak audit failure — value already appears in tracked files or recent git history; store blocked , rotate the cookie in your browser and re-export.
2	Parse error — a cookie file failed the extractor's required- session-cookie check (was not exported from a logged-in browser).
3	Invocation error — bad flag, missing arg, extractor not executable.

Edge cases

- **No cookie files at all.** Exit 0, four SKIP lines in the summary. This is the healthy state before the operator first exports cookies.
- **File exists but empty session cookie.** The delegated extract-tracker-cookies.sh exits 2 (required session cookie not present) → loader reports
PARSE – cookies_<tracker>.txt was not exported from a logged-in session and continues. Re-log into the tracker in the browser and re-export.
- **File exists but no matching domain.** Empty header emitted; safe (writes an empty <T>_COOKIES=' '). Usually means the

operator exported the wrong site's cookies to a file named after the wrong tracker.

- **Leak audit hit.** Exit 1, output names the file/commit + the cookie NAME (never the value). Rotate the cookie in the browser (invalidate the session server-side), re-export, re-run.
- **File is not Netscape TSV.** Parser silently ignores non-conforming lines; the header will be empty and the audit trivially passes. Symptom is a zero-cookie summary — check the file format.
- **~/Downloads does not exist.** Exit 0 with a – nothing to do line. Loader is safe to call in every environment.
- **Two files for one tracker (cookies_rutor.txt + cookies_rutor.txt.bak).** Loader only reads the canonical cookies_<tracker>.txt — backup files ignored.

Internal behaviour

1. **Extractor delegation.** For rutracker, nnmclub, and iptorrents (the private-login trackers the audited primitive teaches, each with a load-bearing required session cookie), calls scripts/extract-tracker-cookies.sh <file> <tracker> and captures the header from stdout, summary from stderr. For rutor and kinozal, the same awk shape is inlined (domain-lowercased, leading-dot stripped, case-insensitive substring on the tracker name, dedup-by-name) — no required-cookie validation for those two (rutor is public/no-login, kinozal's inline path predates the extractor's kinozal support).
2. **Idempotency.** Reads the current .env value for <TRACKER>_COOKIES via _read_env_var, strips surrounding quotes, byte-compares. Match → SKIP.
3. **§11.4.10.A audit.** For every cookie value ≥ 8 chars (and not one of the trivial 1/0/true/false patterns), runs `git grep -F -l -- "$value"` (excluding .env + this loader itself) AND `git log -n 1000 -S"$value"`. Any hit blocks the store.
4. **Atomic write.** `mktemp .env.load-tracker-cookies.XXXXXX`, `chmod 600` BEFORE writing content, then reproduces every non-matching line from .env, appends the new <VAR>='<value>' line, `sync`, `mv -f .env` mode enforced 0600 afterwards. Never leaves a half-written .env.

Related scripts

- **scripts/extract-tracker-cookies.sh** — the single-tracker extraction primitive this loader delegates to. ([Doc](#))
- **scripts/nnmclub-cookie-refresh.sh** — legacy NNM-Club-specific browser cookie refresher; superseded by this loader for the write step.

- **scripts/boba-svc.sh up / restart** — auto-invoke this loader before starting containers.
- **scripts/install.sh** — Stage 6 auto-invokes this loader before boba-svc up.
- **start.sh** — auto-invokes this loader before compose up.
- **challenges/scripts/credentials_wired_challenge.sh** — RED/GREEN guard proving the loader is present + invocable + recognises every file that exists.

Anti-bluff & §11.4 discipline

- **§11.4.10 credentials handling** — cookie **values** never enter logs; every status line prints only tracker, count, names, mtime. .env write is atomic and 0600.
- **§11.4.10.A pre-store leak audit** — mandatory before every write; blocks the store on any hit; operator is instructed to rotate and re-export.
- **§11.4.6 no-guessing** — the loader knows only the closed {rutracker, nmclub, rutor, kinozal, iptorrents} tracker set; unknown trackers are surfaced as --only ignored, never guessed.
- **§11.4.234 always-unblocked** — loader failure never blocks boba-svc up / install.sh / start.sh; a warning surfaces and the boot proceeds with whatever cookies .env already holds.
- **§11.4.115 RED-then-GREEN** — the extended credentials_wired_challenge.sh (checks 7/8/9) is the standing regression guard: strip the loader → the challenge FAILs; restore → PASSes.

Companion of the [Tracker Credentials Manual](#) and the [FAQ](#).